

```
#!/usr/bin/env python
```

```
"""
```

```
run_pipeline.py
```

```
=====
```

PUNTO DE ENTRADA del proyecto.

Orden de arranque (importante):

1. Configurar el logging (stdlib, no necesita nada instalado).
2. Ejecutar el BOOTSTRAP de dependencias (revisa -> instala -> confirma).
3. Solo DESPUES importar los modulos que dependen de pandas/sklearn.

Ese orden es el que permite que el proyecto se ejecute en un entorno limpio: si `pandas` se importara arriba del archivo, el script moriria antes de poder instalarlo.

Uso

```
---  
py run_pipeline.py  
py run_pipeline.py --config config.yaml  
py run_pipeline.py --ruta-dataset datos/panel.csv --columna-target malo \  
                    --columna-id rut --columna-tiempo mes --usar-boruta false  
py run_pipeline.py --sin-autoinstall          (entornos sellados)
```

Todos los flags son opcionales: lo que no se pase se toma de `config.yaml` y, en su defecto, de los valores por defecto del codigo.

```
"""
```

```
from __future__ import annotations
```

```
import argparse  
import logging  
import sys  
from pathlib import Path
```

```
# El paquete vive en src/: se agrega al path para poder ejecutar el script  
# directamente sin necesidad de instalar el proyecto.  
RAIZ = Path(__file__).resolve().parent  
sys.path.insert(0, str(RAIZ / "src"))
```

```
def _a_bool(valor: str | None) -> bool | None:  
    """Convierte un flag de texto a booleano ('true', '1', 'si', 'yes')."""  
    if valor is None:  
        return None  
    return str(valor).strip().lower() in ("true", "1", "si", "sí", "yes", "y", "t")
```

```
def construir_parser() -> argparse.ArgumentParser:  
    """Define la interfaz de linea de comandos."""  
    p = argparse.ArgumentParser(  
        prog="run_pipeline.py",  
        description=(  
            "Pipeline de seleccion de variables para datos de panel: diagnostico, "  
            "univariado, bivariado (IV/Gini), multivariado (correlacion/VIF) y "  
            "Boruta opcional, con bitacora completa en Excel."  
        ),  
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,  
    )  
    p.add_argument("--config", default="config.yaml",  
                   help="Archivo YAML de configuracion.")  
  
    g = p.add_argument_group("Entradas obligatorias (sobrescriben el YAML)")  
    g.add_argument("--ruta-dataset", dest="ruta_dataset")  
    g.add_argument("--columna-target", dest="columna_target")  
    g.add_argument("--columna-id", dest="columna_id")  
    g.add_argument("--columna-tiempo", dest="columna_tiempo")  
    g.add_argument("--usar-boruta", dest="usar_boruta",  
                   help="true / false. Controla la fase 4.")  
    g.add_argument("--ruta-salida-excel", dest="ruta_salida_excel")
```

```

u = p.add_argument_group("Umbrales (opcionales)")
u.add_argument("--umbral-ceros-nulos", dest="umbral_ceros_nulos", type=float)
u.add_argument("--usar-umbral-alterno", dest="usar_umbral_alterno",
               help="true -> aplica el umbral conservador de 90%.")
u.add_argument("--umbral-correlacion", dest="umbral_correlacion", type=float)
u.add_argument("--peso-gini", dest="peso_gini", type=float)
u.add_argument("--peso-iv", dest="peso_iv", type=float)
u.add_argument("--umbral-iv-minimo", dest="umbral_iv_minimo", type=float)
u.add_argument("--umbral-gini-minimo", dest="umbral_gini_minimo", type=float)
u.add_argument("--motor-boruta", dest="motor_boruta",
               choices=["auto", "borutapy", "borutashap", "nativo"])

e = p.add_argument_group("Ejecucion")
e.add_argument("--semilla", dest="semilla", type=int)
e.add_argument("--nivel-log", dest="nivel_log", choices=["DEBUG", "INFO", "WARNING", "ERROR"])
e.add_argument("--sin-autoinstall", action="store_true",
               help="No instala dependencias automaticamente (solo verifica).")
e.add_argument("--generar-demo", action="store_true",
               help="Fuerza la regeneracion del panel sintetico de demostracion.")

return p

```

```

def main() -> int:
    """Arranque completo: logging -> bootstrap -> configuracion -> pipeline."""
    args = construir_parser().parse_args()

    # === 1. Logging =====
    from featsel.logging_utils import configurar_logging

    nivel = getattr(logging, args.nivel_log or "INFO")
    handler_memoria = configurar_logging("outputs/featsel.log", nivel)
    log = logging.getLogger("featsel.main")

    try:
        # === 2. Bootstrap de dependencias =====
        # Se resuelve ANTES de importar nada que dependa de pandas/sklearn.
        from featsel import bootstrap

        usar_boruta = _a_bool(args.usar_boruta)
        if usar_boruta is None:
            # Aun no se leyo el YAML (necesita PyYAML). Se asume True para no
            # dejar la fase 4 sin dependencias; instalar de mas es preferible a
            # tener que reejecutar todo el proceso.
            usar_boruta = True

        reporte_boot = bootstrap.arrancar(
            usar_boruta=usar_boruta,
            autoinstalar=not args.sin_autoinstall,
        )

        # === 3. Configuracion =====
        from featsel.config import cargar_config

        overrides = {
            "ruta_dataset": args.ruta_dataset,
            "columna_target": args.columna_target,
            "columna_id": args.columna_id,
            "columna_tiempo": args.columna_tiempo,
            "usar_boruta": _a_bool(args.usar_boruta),
            "ruta_salida_excel": args.ruta_salida_excel,
            "umbral_ceros_nulos": args.umbral_ceros_nulos,
            "usar_umbral_alterno": _a_bool(args.usar_umbral_alterno),
            "umbral_correlacion": args.umbral_correlacion,
            "peso_gini": args.peso_gini,
            "peso_iv": args.peso_iv,
            "umbral_iv_minimo": args.umbral_iv_minimo,
            "umbral_gini_minimo": args.umbral_gini_minimo,
            "motor_boruta": args.motor_boruta,
            "semilla": args.semilla,
            "nivel_log": args.nivel_log,

```

```

}
cfg = cargar_config(args.config, overrides)

# === 4. Dataset de demostracion (si corresponde) =====
ruta_datos = Path(cfg.ruta_dataset)
if args.generar_demo or (not ruta_datos.is_file() and cfg.generar_demo_si_falta):
    log.warning(
        "El dataset '%s' no existe%s. Se genera un panel sintetico de demostracion "
        "usando los nombres de columna de la configuracion.",
        ruta_datos, " (regeneracion forzada)" if args.generar_demo else "",
    )
    from generar_datos_demo import generar_panel_demo

    generar_panel_demo(cfg)

# === 5. Pipeline =====
from featsel.pipeline import ejecutar

resultados = ejecutar(cfg, handler_log=handler_memoria, reporte_bootstrap=reporte_boot)

# === 6. Resumen en consola =====
sel = resultados["variables_seleccionadas"]
print("\n" + "=" * 78)
print(" SELECCION DE VARIABLES COMPLETADA")
print("=" * 78)
print(f" Dataset           : {cfg.ruta_dataset}")
print(f" Target / id / t    : {cfg.columna_target} / {cfg.columna_id} / {cfg.columna_tiempo}")
print(f" Boruta              : {'SI (' + resultados['boruta_meta'].get('motor', '') + ') ' if resultados['boruta_meta'].get('motor', '') != '' else ''}")
print(f" Duracion            : {resultados['segundos']:.2f} s")
print(f" Variables finales: {len(sel)}")
for i, v in enumerate(sel, start=1):
    print(f"      {i:>3}. {v}")
print(f"\n Bitacora Excel    : {resultados['ruta_excel']}")
if resultados.get("ruta_dataset_final"):
    print(f" Dataset final      : {resultados['ruta_dataset_final']}")
print("=" * 78 + "\n")
return 0

except Exception as exc: # noqa: BLE001 - frontera del programa
    log.exception("El pipeline termino con error: %s", exc)
    print(f"\nERROR: {exc}\n", file=sys.stderr)
    return 1

if __name__ == "__main__":
    raise SystemExit(main())

```